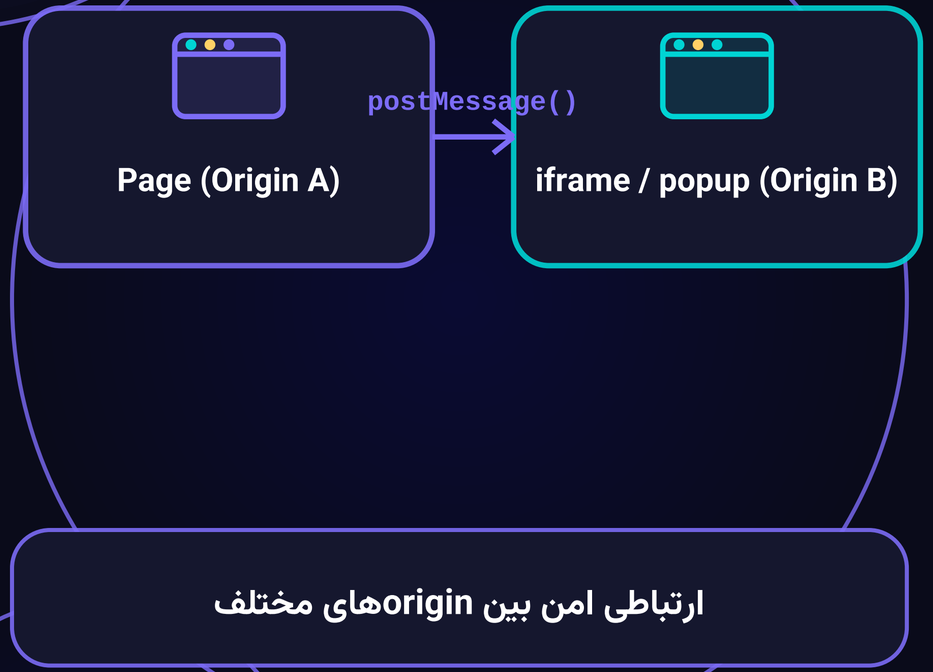
postMessage چیست؟

farnoodReact | farnoodlotfali

postMessage چیست؟

postMessage یک متد است که ارتباط cross-origin بین Window objectها را ممکن می‌کند.

برای مثال، بین یک صفحه و popup ای که باز کرده، یا بین یک صفحه و iframe ای که داخلش embed شده است.



به‌طور معمول، اسکریپت‌های دو صفحه‌ی مختلف تنها زمانی به هم دسترسی دارند که origin یکسانی داشته باشند:

- Port
- Host
- Protocol

چرا به postMessage نیاز داریم؟

same-origin policy جلوی دسترسی مستقیم اسکریپت‌های یک صفحه به صفحه‌ای با origin متفاوت را می‌گیرد. postMessage راهی امن و صریح برای عبور از این دیوار است.

مزایای کلیدی

Cross-Origin

یک صفحه می‌تواند با iframe یا popup روی origin کاملا متفاوتی صحبت کند.

Controlled

گیرنده پیش از اعتماد به هر پیام، origin فرستنده را بررسی می‌کند.

Two-Way

هر دو پنجره می‌توانند بفرستند و بگیرند؛ یعنی ارتباط واقعا دوطرفه است.

مفاهیم کلیدی

بخش‌هایی که هر بار با postMessage کار می‌کنید، آن‌ها را به‌کار می‌برید





message

داده‌ای که می‌خواهید به پنجره‌ی دیگر بفرستید.

```
win.postMessage(data, origin)
```



targetOrigin

originی که پنجره‌ی گیرنده باید داشته باشد، برای امنیت.

```
"https://example.com"
```



message event

روی پنجره‌ی گیرنده، هنگام رسیدن پیام رخ می‌دهد.

```
addEventListener("message", fn)
```



event.origin

origin فرستنده؛ همیشه پیش از اعتماد بررسی‌اش کنید.

```
if (e.origin !== trusted)
```



مثال: ارسال پیام

فرض کنید صفحه‌ای یک iframe دارد. صفحه‌ی والد با ذکر دقیق origin، پیامی به آن iframe می‌فرستد.

parent.js

```
const frame = document.getElementById("myFrame");

frame.contentWindow.postMessage(
  { type: "greet", text: "Hello iframe!" },
  "https://child.example.com"
);
```

popup.js

```
// Sending to a popup window works the same way
const popup = window.open("https://child.example.com");

popup.postMessage(
  { type: "greet", text: "Hello popup!" },
  "https://child.example.com"
);
```

targetOrigin باید دقیقا با origin گیرنده یکی باشد، وگرنه مرورگر پیام را تحویل نمی‌دهد.

مثال: دریافت پیام

پنجره‌ی گیرنده به message event گوش می‌دهد و همیشه باید منبع پیام را تأیید کند.

child.js

```
window.addEventListener("message", (event) => {
  if (event.origin !== "https://parent.example.com") {
    return; // ignore untrusted senders
  }

  console.log(event.data);
  // { type: "greet", text: "Hello iframe!" }
});
```

reply.js

```
// Replying back to the sender
window.addEventListener("message", (event) => {
  if (event.origin !== "https://parent.example.com") return;

  event.source.postMessage(
    { type: "ack", text: "Got it!" },
    event.origin
  );
});
```

event.origin می‌گوید پیام واقعا از کجا آمده؛ هرگز فقط به event.data اعتماد نکنید.

Same-Origin در برابر postMessage

سناریو	صفحات Same-Origin	صفحات Cross-Origin
دسترسی مستقیم JS	مجاز (window.frame.var)	توسط مرورگر مسدود می‌شود
اشتراک‌گذاری داده	خواندن متغیرهای یکدیگر	باید از postMessage استفاده شود
بررسی امنیتی	ضمنی، به دلیل origin یکسان	باید خودتان event.origin را چک کنید
مورد استفاده‌ی رایج	iframeهای هم‌سایت	ویجت‌های شخص ثالث، iframe پرداخت، OAuth

اگر origin یکسان است، دسترسی مستقیم کار می‌کند؛ در غیر این صورت سراغ postMessage بروید.

نکات کلیدی

۱

Cross-origin Communication

postMessage به Window objectها اجازه می‌دهد امن بین originهای مختلف صحبت کنند.

۲

Same-origin Policy

اسکریپت‌ها فقط وقتی protocol، host و port یکسان باشد، مستقیم به هم دسترسی دارند.

۳

همیشه origin را چک کنید

پیش از اعتماد به هر پیام، event.origin را در سمت گیرنده تأیید کنید.

۴

موارد استفاده‌ی رایج

iframeها، popupها، فرآیندهای OAuth، و ویجت‌های شخص ثالث embed شده.